

Assignment – 3

1. Small E-commerce Backend (orders, products, carts)

Database.java

```
import java.sql.*;

public class Database {

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection("jdbc:sqlite:ecommerce.db");
    }

    public static void init() throws SQLException {
        try (Connection conn = getConnection(); Statement st = conn.createStatement()) {

            st.executeUpdate("""
                CREATE TABLE IF NOT EXISTS products(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT,
                    price REAL,
                    stock INTEGER
                );
            """);

            st.executeUpdate("""
                CREATE TABLE IF NOT EXISTS users(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    username TEXT,
                    email TEXT
                );
            """);

            st.executeUpdate("""
                CREATE TABLE IF NOT EXISTS carts(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    user_id INTEGER
                );
            """);

            st.executeUpdate("""
                CREATE TABLE IF NOT EXISTS cart_items(
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    cart_id INTEGER,
                    product_id INTEGER,
                    qty INTEGER
                );
            """);
```

```

st.executeUpdate("""
CREATE TABLE IF NOT EXISTS orders(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    status TEXT,
    total REAL,
    created_at TEXT
);
""");

st.executeUpdate("""
CREATE TABLE IF NOT EXISTS order_items(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER,
    product_id INTEGER,
    qty INTEGER,
    price REAL
);
""");
}
}
}

```

EcommerceService.java

```

import java.sql.*;
import java.time.LocalDateTime;

public class EcommerceService {

    // Add product to cart
    public static void addToCart(int userId, int productId, int qty) throws Exception {
        try (Connection conn = Database.getConnection()) {

            // find or create cart for user
            int cartId = -1;
            PreparedStatement ps = conn.prepareStatement("SELECT id FROM carts WHERE user_id=?");
            ps.setInt(1, userId);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) cartId = rs.getInt(1);

            if (cartId == -1) {
                ps = conn.prepareStatement("INSERT INTO carts(user_id) VALUES(?)",
Statement.RETURN_GENERATED_KEYS);
                ps.setInt(1, userId);
                ps.executeUpdate();
                rs = ps.getGeneratedKeys();
                if (rs.next()) cartId = rs.getInt(1);
            }

```

```

// insert cart item
ps = conn.prepareStatement("INSERT INTO cart_items(cart_id,product_id,qty) VALUES(?,?,?)");
ps.setInt(1, cartId);
ps.setInt(2, productId);
ps.setInt(3, qty);
ps.executeUpdate();

System.out.println("Added to cart!");
}
}

```

```

// Checkout – deduct stock and create order
public static void checkout(int userId) throws Exception {
    try (Connection conn = Database.getConnection()) {
        conn.setAutoCommit(false);

        try {
            // fetch cart items
            PreparedStatement ps = conn.prepareStatement("""
                SELECT ci.product_id, ci.qty, p.price, p.stock
                FROM cart_items ci
                JOIN carts c ON ci.cart_id=c.id
                JOIN products p ON ci.product_id=p.id
                WHERE c.user_id=?
            """);
            ps.setInt(1, userId);
            ResultSet rs = ps.executeQuery();

            double total = 0;
            boolean empty = true;

            while (rs.next()) {
                empty = false;
                int productId = rs.getInt("product_id");
                int qty = rs.getInt("qty");
                int stock = rs.getInt("stock");

                if (qty > stock) {
                    conn.rollback();
                    System.out.println("Not enough stock!");
                    return;
                }

                // deduct stock
                PreparedStatement upd = conn.prepareStatement(
                    "UPDATE products SET stock = stock - ? WHERE id = ?"
                );
            }

```

```

        upd.setInt(1, qty);
        upd.setInt(2, productId);
        upd.executeUpdate();

        total += qty * rs.getDouble("price");
    }

    if (empty) {
        System.out.println("Cart is empty!");
        return;
    }

    // create order
    ps = conn.prepareStatement("""
        INSERT INTO orders(user_id,status,total,created_at)
        VALUES(?,?,?,?)
        """, Statement.RETURN_GENERATED_KEYS);

    ps.setInt(1, userId);
    ps.setString(2, "COMPLETED");
    ps.setDouble(3, total);
    ps.setString(4, LocalDateTime.now().toString());
    ps.executeUpdate();

    rs = ps.getGeneratedKeys();
    int orderId = rs.next() ? rs.getInt(1) : -1;

    // insert order items
    ps = conn.prepareStatement("""
        INSERT INTO order_items(order_id, product_id, qty, price)
        SELECT ?, ci.product_id, ci.qty, p.price
        FROM cart_items ci
        JOIN carts c ON ci.cart_id=c.id
        JOIN products p ON ci.product_id=p.id
        WHERE c.user_id=?
        """);
    ps.setInt(1, orderId);
    ps.setInt(2, userId);
    ps.executeUpdate();

    // clear cart
    ps = conn.prepareStatement("""
        DELETE FROM cart_items
        WHERE cart_id = (SELECT id FROM carts WHERE user_id=?)
        """);
    ps.setInt(1, userId);
    ps.executeUpdate();

```

```

        conn.commit();
        System.out.println("Checkout successful! Order ID = " + orderId);

    } catch (Exception e) {
        conn.rollback();
        throw e;
    }
}
}
}
}

```

Main.java

```

public class Main {
    public static void main(String[] args) throws Exception {

        // create DB & tables
        Database.init();

        try (var conn = Database.getConnection()) {
            conn.createStatement().executeUpdate(
                "INSERT INTO products(name, price, stock) VALUES ('Pen', 10, 100)"
            );
            conn.createStatement().executeUpdate(
                "INSERT INTO users(username,email) VALUES ('alice','a@mail.com')"
            );
            conn.createStatement().executeUpdate(
                "INSERT INTO carts(user_id) VALUES (1)"
            );
        }

        // Add items
        EcommerceService.addToCart(1, 1, 4);

        // Checkout
        EcommerceService.checkout(1);
    }
}

```

```

Added to cart!
Checkout successful! Order ID = 1
BUILD SUCCESSFUL (total time: 0 seconds)
||

```

2. Student Attendance & Timetable System (reports + analytics)

Database.java

```
import java.sql.*;
```

```
public class Database {
```

```
    public static Connection getConnection() throws SQLException {  
        return DriverManager.getConnection("jdbc:sqlite:attendance.db");  
    }
```

```
    public static void init() throws SQLException {  
        try (Connection conn = getConnection(); Statement st = conn.createStatement()) {
```

```
            st.executeUpdate("""  
                CREATE TABLE IF NOT EXISTS students(  
                    id INTEGER PRIMARY KEY AUTOINCREMENT,  
                    name TEXT,  
                    roll TEXT  
                );  
            """);
```

```
            st.executeUpdate("""  
                CREATE TABLE IF NOT EXISTS classes(  
                    id INTEGER PRIMARY KEY AUTOINCREMENT,  
                    name TEXT  
                );  
            """);
```

```
            st.executeUpdate("""  
                CREATE TABLE IF NOT EXISTS subjects(  
                    id INTEGER PRIMARY KEY AUTOINCREMENT,  
                    class_id INTEGER,  
                    name TEXT  
                );  
            """);
```

```
            st.executeUpdate("""  
                CREATE TABLE IF NOT EXISTS attendance(  
                    id INTEGER PRIMARY KEY AUTOINCREMENT,  
                    student_id INTEGER,  
                    subject_id INTEGER,  
                    date TEXT,  
                    status TEXT,  
                    UNIQUE(student_id, subject_id, date)  
                );  
            """);
```

```
}
```

```
}  
}
```

AttendanceService.java

```
import java.sql.*;
```

```
public class AttendanceService {
```

```
    // Insert student
```

```
    public static void addStudent(String name, String roll) throws Exception {
```

```
        try (Connection conn = Database.getConnection()) {  
            PreparedStatement ps = conn.prepareStatement(  
                "INSERT INTO students(name, roll) VALUES(?,?)"  
            );  
            ps.setString(1, name);  
            ps.setString(2, roll);  
            ps.executeUpdate();  
        }  
    }
```

```
    // Insert class
```

```
    public static void addClass(String name) throws Exception {
```

```
        try (Connection conn = Database.getConnection()) {  
            conn.createStatement().executeUpdate(  
                "INSERT INTO classes(name) VALUES('" + name + "')"  
            );  
        }  
    }
```

```
    // Insert subject
```

```
    public static void addSubject(int classId, String name) throws Exception {
```

```
        try (Connection conn = Database.getConnection()) {  
            PreparedStatement ps = conn.prepareStatement(  
                "INSERT INTO subjects(class_id,name) VALUES(?,?)"  
            );  
            ps.setInt(1, classId);  
            ps.setString(2, name);  
            ps.executeUpdate();  
        }  
    }
```

```
    // Mark attendance
```

```
    public static void markAttendance(int studentId, int subjectId, String date, String status)  
        throws Exception {
```

```
        try (Connection conn = Database.getConnection()) {  
            PreparedStatement ps = conn.prepareStatement(""  
                INSERT INTO attendance(student_id,subject_id,date,status)
```

```

VALUES(?,?,?,?)
""");

ps.setInt(1, studentId);
ps.setInt(2, subjectId);
ps.setString(3, date);
ps.setString(4, status);

try {
    ps.executeUpdate();
    System.out.println("Attendance marked.");
} catch (SQLException e) {
    System.out.println("Attendance already exists for this date!");
}
}
}

// Attendance percentage
public static void attendancePercentage(int studentId) throws Exception {
    try (Connection conn = Database.getConnection()) {

        PreparedStatement ps = conn.prepareStatement("""
        SELECT
            SUM(CASE WHEN status='P' THEN 1 ELSE 0 END) AS present_days,
            COUNT(*) AS total_days
        FROM attendance
        WHERE student_id=?
        """);

        ps.setInt(1, studentId);

        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            int present = rs.getInt("present_days");
            int total = rs.getInt("total_days");

            if (total == 0) {
                System.out.println("No attendance yet.");
                return;
            }

            double percent = (present * 100.0) / total;
            System.out.println("Attendance % = " + percent);
        }
    }
}
}

```



```
// Show defaulters (<75%)
public static void showDefaulters() throws Exception {
    try (Connection conn = Database.getConnection()) {

        PreparedStatement ps = conn.prepareStatement("""
            SELECT s.id, s.name,
                SUM(CASE WHEN a.status='P' THEN 1 ELSE 0 END)*100.0 / COUNT(a.id) AS percent
            FROM students s
            JOIN attendance a ON s.id = a.student_id
            GROUP BY s.id
            HAVING percent < 75
            """);

        ResultSet rs = ps.executeQuery();

        System.out.println("DEFAULTERS (<75%):");
        while (rs.next()) {
            System.out.println(rs.getInt("id") + " - " +
                rs.getString("name") + " - " +
                rs.getDouble("percent") + "%");
        }
    }
}
}
```

Main.java

```
public class Main {
    public static void main(String[] args) throws Exception {

        Class.forName("org.sqlite.JDBC"); // load driver
        Database.init();

        // sample data
        AttendanceService.addStudent("Ananya", "101");
        AttendanceService.addStudent("Bhuvan", "102");

        AttendanceService.addClass("Class A");
        AttendanceService.addSubject(1, "Math");

        // mark attendance (P = present, A = absent)
        AttendanceService.markAttendance(1, 1, "2025-01-01", "P");
        AttendanceService.markAttendance(1, 1, "2025-01-02", "A");
        AttendanceService.markAttendance(1, 1, "2025-01-03", "P");

        AttendanceService.markAttendance(2, 1, "2025-01-01", "A");
        AttendanceService.markAttendance(2, 1, "2025-01-02", "A");
        AttendanceService.markAttendance(2, 1, "2025-01-03", "A");
    }
}
```

```

System.out.println("\nPercentage for Ananya:");
AttendanceService.attendancePercentage(1);

System.out.println("\nPercentage for Bhuvan:");
AttendanceService.attendancePercentage(2);

System.out.println("\nDefaulters:");
AttendanceService.showDefaulters();
}
}

```

```

Attendance marked.
Attendance marked.
Attendance marked.
Attendance marked.
Attendance marked.
Attendance marked.

```

```

Percentage for Ananya:
Attendance % = 66.66666666666667

```

```

Percentage for Bhuvan:
Attendance % = 0.0

```

```

Defaulters:
DEFAULTERS (<75%):
1 - Ananya - 66.66666666666667%
2 - Bhuvan - 0.0%
BUILD SUCCESSFUL (total time: 0 seconds)

```

3. Appointment Booking System (time-slot conflicts)

```

Database.java
import java.sql.*;

public class Database {

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection("jdbc:sqlite:appointments.db");
    }

    public static void init() throws SQLException {
        try (Connection conn = getConnection(); Statement st = conn.createStatement()) {

            st.executeUpdate("""

```

```

CREATE TABLE IF NOT EXISTS providers(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    specialty TEXT
);
""");

st.executeUpdate("""
CREATE TABLE IF NOT EXISTS timeslots(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    provider_id INTEGER,
    start_ts TEXT,
    end_ts TEXT
);
""");

st.executeUpdate("""
CREATE TABLE IF NOT EXISTS bookings(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timeslot_id INTEGER,
    user_id INTEGER,
    status TEXT,
    created_at TEXT
);
""");
}
}
}

AppoitnmentService.java
import java.sql.*;
import java.time.LocalDateTime;

public class AppointmentService {

    // Add provider (doctor)
    public static void addProvider(String name, String specialty) throws Exception {
        try (Connection conn = Database.getConnection()) {
            PreparedStatement ps = conn.prepareStatement("""
                INSERT INTO providers(name, specialty)
                VALUES (?, ?)
            """);
            ps.setString(1, name);
            ps.setString(2, specialty);
            ps.executeUpdate();
        }
    }
}

```

```
// Add timeslot
public static void addTimeSlot(int providerId, String start, String end) throws Exception {
    try (Connection conn = Database.getConnection()) {
        PreparedStatement ps = conn.prepareStatement("""
            INSERT INTO timeslots(provider_id, start_ts, end_ts)
            VALUES (?, ?, ?)
            """);
        ps.setInt(1, providerId);
        ps.setString(2, start);
        ps.setString(3, end);
        ps.executeUpdate();
    }
}
```

```
// Book a timeslot with conflict detection
public static void bookAppointment(int userId, int slotId) throws Exception {
    try (Connection conn = Database.getConnection()) {
        conn.setAutoCommit(false);

        // Check if slot is already booked
        PreparedStatement check = conn.prepareStatement("""
            SELECT COUNT(*) AS c
            FROM bookings
            WHERE timeslot_id=? AND status='BOOKED'
            """);
        check.setInt(1, slotId);
        ResultSet rs = check.executeQuery();

        if (rs.next() && rs.getInt("c") > 0) {
            conn.rollback();
            System.out.println("Slot already booked!");
            return;
        }
    }
}
```

```
// Book slot
PreparedStatement ps = conn.prepareStatement("""
    INSERT INTO bookings(timeslot_id, user_id, status, created_at)
    VALUES (?, ?, ?, ?)
    """);

ps.setInt(1, slotId);
ps.setInt(2, userId);
ps.setString(3, "BOOKED");
ps.setString(4, LocalDateTime.now().toString());
ps.executeUpdate();

conn.commit();
System.out.println("Appointment booked!");
```

```
}  
}
```

// Cancel booking

```
public static void cancelBooking(int bookingId) throws Exception {  
    try (Connection conn = Database.getConnection()) {  
        PreparedStatement ps = conn.prepareStatement("""  
            UPDATE bookings SET status='CANCELLED' WHERE id=?  
            """);  
        ps.setInt(1, bookingId);  
        ps.executeUpdate();  
  
        System.out.println("Appointment cancelled.");  
    }  
}
```

// View all available slots for provider

```
public static void showAvailableSlots(int providerId) throws Exception {  
    try (Connection conn = Database.getConnection()) {
```

// Fetch provider details

```
    PreparedStatement providerPS = conn.prepareStatement("""  
        SELECT name, specialty  
        FROM providers  
        WHERE id = ?  
        """);  
    providerPS.setInt(1, providerId);  
    ResultSet prs = providerPS.executeQuery();
```

```
    if (prs.next()) {  
        System.out.println("Provider: " + prs.getString("name") +  
            " (" + prs.getString("specialty") + ")");  
    } else {  
        System.out.println("Provider not found!");  
        return;  
    }
```

// Fetch available slots

```
    PreparedStatement ps = conn.prepareStatement("""  
        SELECT t.id, t.start_ts, t.end_ts  
        FROM timeslots t  
        LEFT JOIN bookings b  
            ON t.id = b.timeslot_id AND b.status='BOOKED'  
        WHERE t.provider_id=? AND b.id IS NULL  
        """);
```

```
    ps.setInt(1, providerId);  
    ResultSet rs = ps.executeQuery();
```

```

System.out.println("Available Slots:");
boolean found = false;

while (rs.next()) {
    found = true;
    System.out.println("Slot ID: " + rs.getInt("id") +
        " Time: " + rs.getString("start_ts") +
        " - " + rs.getString("end_ts"));
}

if (!found) {
    System.out.println("No available slots.");
}
}
}
}

```

Main.java

```

public class Main {
    public static void main(String[] args) throws Exception {

        Class.forName("org.sqlite.JDBC");
        Database.init();

        // sample data
        AppointmentService.addProvider("Dr. Smith", "Cardiologist");
        AppointmentService.addTimeSlot(1, "2025-01-10 10:00", "2025-01-10 10:30");
        AppointmentService.addTimeSlot(1, "2025-01-10 10:30", "2025-01-10 11:00");

        // show available
        AppointmentService.showAvailableSlots(1);

        // book a slot
        AppointmentService.bookAppointment(101, 1);

        // try booking same slot again
        AppointmentService.bookAppointment(102, 1); // should show conflict

        // cancel booking
        AppointmentService.cancelBooking(1);

    }
}

```

Provider: Dr. Smith (Cardiologist)

Available Slots:

Slot ID: 2 Time: 2025-01-10 10:30 - 2025-01-10 11:00

Slot ID: 3 Time: 2025-01-10 10:00 - 2025-01-10 10:30

Slot ID: 4 Time: 2025-01-10 10:30 - 2025-01-10 11:00

Slot ID: 5 Time: 2025-01-10 10:00 - 2025-01-10 10:30

Slot ID: 6 Time: 2025-01-10 10:30 - 2025-01-10 11:00

Slot already booked!

Slot already booked!

? Appointment cancelled.

BUILD SUCCESSFUL (total time: 0 seconds)